

Ride Sharing Analytics & Driver Performance pipeline

A

Project Report

Submitted By

Name: Khushi Singh

Student ID: CT_CSI_DE_86

Course: B.Tech (Computer Science & Engineering)

College: Maharishi Markandeshwar Deemed to be University, Mullana

Submission Date: 11/07/2026

1.Project Overview

The Ride Sharing Data Engineering Pipeline is a PySpark-based project that processes ride-sharing data using the Medallion Architecture (Bronze, Silver, and Gold). It uses three datasets: Drivers, Trips, and Trip Logs. The Bronze layer ingests raw CSV data and stores it in Parquet format. The Silver layer cleans the data, removes duplicates, joins the datasets, and creates new columns such as Trip Duration, Completion Status, and Driver Performance. The Gold layer generates business insights, including Total Revenue, Top Drivers, Revenue by City, Cancellation Analysis, and Popular Pickup Locations. This project demonstrates how raw data can be transformed into clean and meaningful information for business analysis using PySpark.

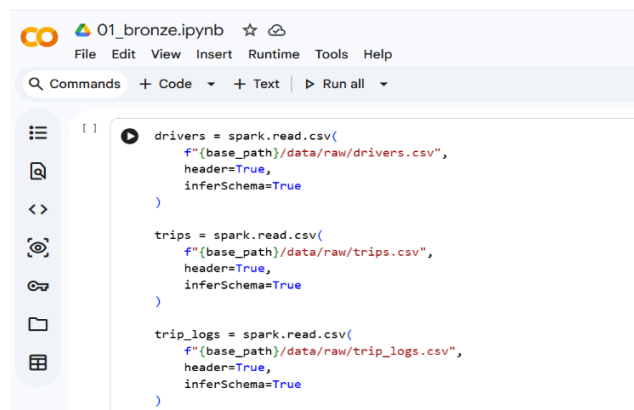
2. Technologies Used

- Python
- PySpark
- Google Colab
- Google Drive
- Git & GitHub

3. Project Workflow

i. Bronze Layer

- Reads raw CSV files using PySpark.
- Converts raw data into Parquet format.
- Stores data in the Bronze layer.



```
drivers = spark.read.csv(
    f"{base_path}/data/raw/drivers.csv",
    header=True,
    inferSchema=True
)

trips = spark.read.csv(
    f"{base_path}/data/raw/trips.csv",
    header=True,
    inferSchema=True
)

trip_logs = spark.read.csv(
    f"{base_path}/data/raw/trip_logs.csv",
    header=True,
    inferSchema=True
)
```

01_bronze.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```

Number of Drivers : 150
Number of Trips : 150
Number of Trip Logs : 150

[ ]
1 drivers.write.mode("overwrite").parquet(
    f"{base_path}/data/bronze/drivers"
)

trips.write.mode("overwrite").parquet(
    f"{base_path}/data/bronze/trips"
)

trip_logs.write.mode("overwrite").parquet(
    f"{base_path}/data/bronze/trip_logs"
)

print("Bronze Layer Created Successfully!")

... Bronze Layer Created Successfully!

```

ii.Silver Layer

- Reads Bronze data.
- Removes duplicates and handles missing values.
- Joins the datasets.
- Calculates Trip Duration.
- Creates Completion Status and Driver Performance columns.
- Stores cleaned data in the Silver layer.

02_silver.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```

only showing top 5 rows

[ ]
1 driver_trip = trips.join(
    drivers,
    on="driver_id",
    how="inner"
)

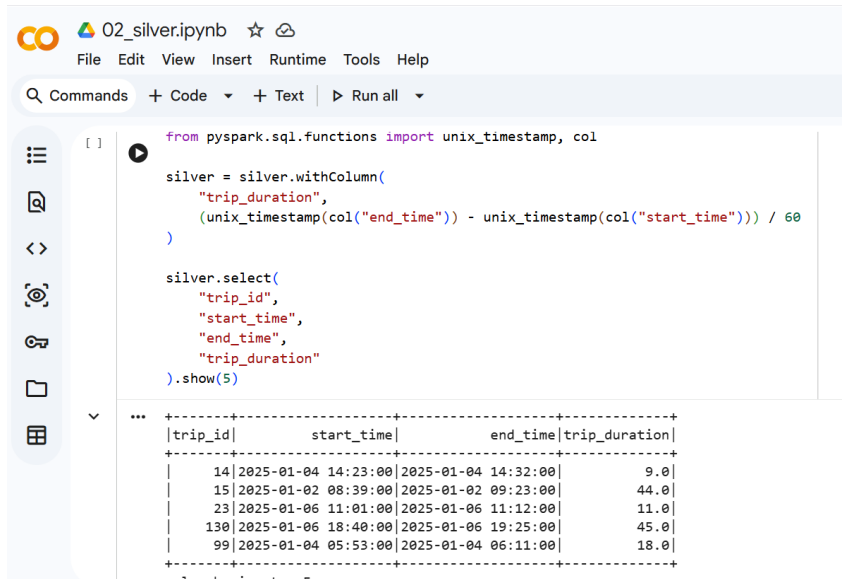
driver_trip.show(5)

...

```

driver_id	trip_id	pickup_location	drop_location	distance_km	fare_amount	trip_status	name	city	rating
99	14	IT Park	IT Park	17.96	232.49	Completed	Rahul_99	Delhi	4.2
31	72	City Center	Mall	16.9	0.0	Cancelled	Neha_31	Mumbai	4.6
85	15	IT Park	IT Park	14.48	157.98	Completed	Karan_85	Mumbai	5.0
44	88	City Center	Railway Station	6.03	0.0	Cancelled	Karan_44	Pune	4.6
65	23	Mall	Railway Station	21.2	239.39	Completed	Amit_65	Bangalore	3.6

only showing top 5 rows

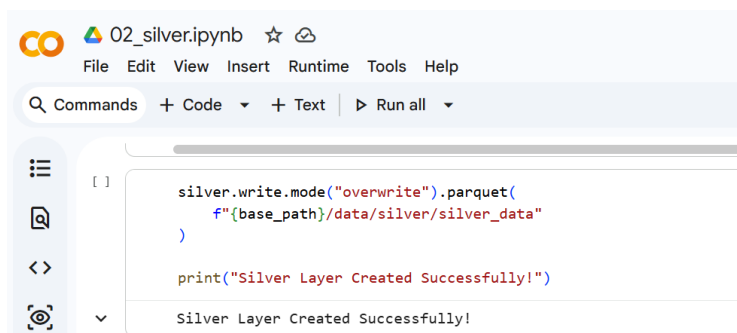


```
from pyspark.sql.functions import unix_timestamp, col

silver = silver.withColumn(
    "trip_duration",
    (unix_timestamp(col("end_time")) - unix_timestamp(col("start_time"))) / 60
)

silver.select(
    "trip_id",
    "start_time",
    "end_time",
    "trip_duration"
).show(5)
```

trip_id	start_time	end_time	trip_duration
14	2025-01-04 14:23:00	2025-01-04 14:32:00	9.0
15	2025-01-02 08:39:00	2025-01-02 09:23:00	44.0
23	2025-01-06 11:01:00	2025-01-06 11:12:00	11.0
130	2025-01-06 18:40:00	2025-01-06 19:25:00	45.0
99	2025-01-04 05:53:00	2025-01-04 06:11:00	18.0



```
silver.write.mode("overwrite").parquet(
    f"{base_path}/data/silver/silver_data"
)

print("Silver Layer Created Successfully!")
```

Silver Layer Created Successfully!

iii. Gold Layer

- Reads Silver data.
- Performs business analysis.
- Calculates Total Revenue, Top Drivers, Revenue by City, Cancellation Analysis, and Popular Pickup Locations.
- Stores the final business-ready data in the Gold layer.

03_gold.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[]

▶

```
driver_revenue = silver.groupBy(
    "driver_id",
    "name"
).agg(
    round(sum("fare_amount"),2).alias("Total_Revenue")
).orderBy(
    col("Total_Revenue").desc()
)

driver_revenue.show()
```

...

driver_id	name	Total_Revenue
114	Neha_114	720.44
107	Rahul_107	530.96
65	Amit_65	493.75
102	Vikas_102	390.52
132	Priya_132	387.74
76	Priya_76	369.47
20	Vikas_20	336.66
63	Amit_63	334.21

03_gold.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[]

▶

```
top_drivers = driver_revenue.limit(10)

top_drivers.show()
```

...

driver_id	name	Total_Revenue
114	Neha_114	720.44
107	Rahul_107	530.96
65	Amit_65	493.75
102	Vikas_102	390.52
132	Priya_132	387.74
76	Priya_76	369.47
20	Vikas_20	336.66
63	Amit_63	334.21
112	Amit_112	326.4
103	Vikas_103	256.07

125	Neha_125	214.39
54	Anjali_54	212.08
16	Anjun_16	211.15

only showing top 20 rows

03_gold.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[]

▶

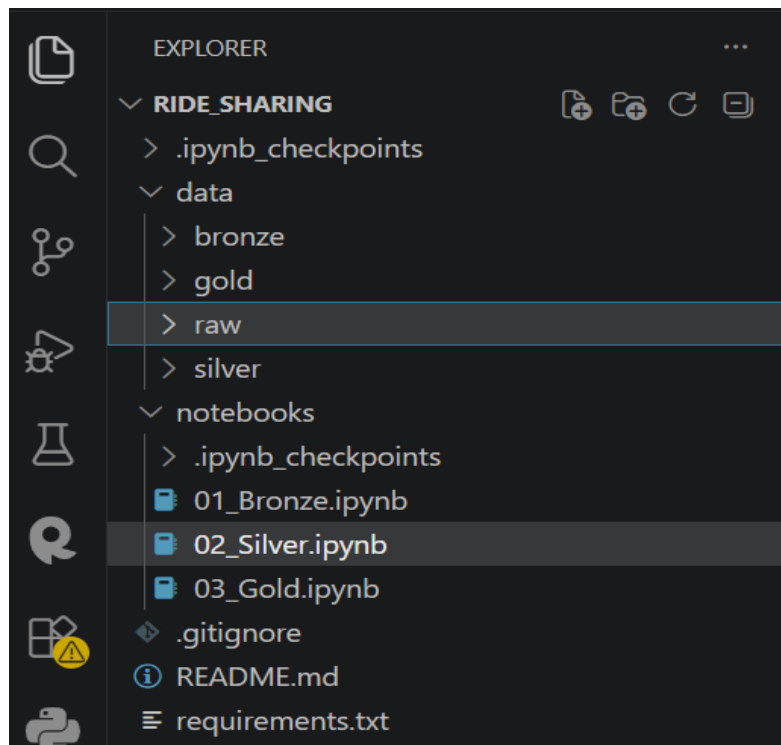
```
gold.write.mode("overwrite").parquet(
    f"{base_path}/data/gold/gold_data"
)

print("Gold Layer Created Successfully!")
```

...

Gold Layer Created Successfully!

4. Project Structure



5. Conclusion

This project successfully implements the Medallion Architecture using PySpark. The raw data is transformed into clean and structured datasets, and the Gold layer provides meaningful business insights that can support decision-making in a ride-sharing system.